



中国矿业大学
CHINA UNIVERSITY OF MINING AND TECHNOLOGY

《大数据可视化》综合设计报告二

题目：智慧城市交通流量实时监控大屏

学 院 计算机科学与技术学院

班 级 2023 级大数据一班

姓 名 王志豪

学 号 08232337

中国矿业大学

2026 年 03 月

课程设计报告格式要求：

1. 报告中应包含一、二级标题；
2. 正文宋体小 4 号，1.5 倍行距；
3. 报告正文不少于 1000 字；
4. 上交报告电子版：<https://v.wjx.cn/vm/mv3f28U.aspx>，截止 4 月 26 日。

课程设计报告评分要点：

1. 报告中可视化技术的应用情况（20%）
2. 报告中可视化应用功能有效及完整性（40%）
3. 报告中的个人总结及体会情况（20%）
4. 报告格式规范程度（20%）

评语：

评分：

日期：

目录

1.背景及功能概要.....	3
1.1 背景.....	3
1.2 功能概要.....	3
1.3 项目结构与使用指南	4
2.可视化系统设计.....	6
2.1 概要设计.....	6
2.2 可视化设计.....	8
3.可视化实现及验证	13
3.1 代码分析.....	13
3.2 运行截图.....	22
4.总结及体会.....	24

1. 背景及功能概要

1.1 背景

随着城市机动车保有量持续增长，交通拥堵、通行效率下降和高峰期路网压力失衡等问题日益突出。传统交通管理方式往往依赖事后统计和分散监控，难以在短时间内形成对全局路况的直观判断，也难以为管理者和公众提供及时、可执行的出行建议。因此，建设一个集实时感知、可视化展示与研判辅助于一体的交通监测平台，已成为智慧城市建设中的重要方向。

本项目面向课程设计场景，以徐州市路网为业务对象，构建了一个“数据采集与模拟、趋势分析、预警研判、路径建议、智能问答”一体化系统。项目在保证可演示性和工程完整性的同时，强调交通业务逻辑与数据可解释性，既可用于展示智慧交通大屏的核心能力，也为后续接入真实外部数据源和智能调度策略预留了扩展空间。

1.2 功能概要

(1). 实时交通监控与地图联动

系统通过前端的 HTML/CSS/JavaScript 与 ECharts 构建可视化大屏，实时展示路段流量、速度和拥堵指数，并在徐州行政区地图上进行联动渲染；后端使用 FastAPI 提供实时接口，前端通过 Fetch 周期拉取数据，实现秒级刷新与态势总览。

(2). 历史趋势分析与时序展示

后端基于 APScheduler 定时采样，将路段运行数据按固定粒度写入 MySQL，使用 SQLAlchemy 完成历史数据查询与聚合；前端利用 ECharts 折线图和统计组件展示 24 小时趋势变化，形成“当前状态 + 历史轨迹”的分析视角。

(3). 交通预警与风险研判

预警模块由 Python 业务算法实现，综合占有率、流量负载率、速度下降幅度和短时趋势进行风险评分，并分级输出预警结果；服务层通过 FastAPI + Pydantic 返回结构化数据，前端以卡片化方式呈现风险原因与处置建议，提升可解释性与可执行性。

(4). 绕行推荐与路径决策

系统在后端构建区县路网图模型，结合实时拥堵压力进行路径枚举与打分排序，生成“优先、平衡、备用”三类绕行方案；前端通过交互式起终点选择与推荐列表展示，让用户快速完成路线比较与决策，体现图算法与业务规则融合的能力。

(5). AI 交通问答服务

AI 模块在 FastAPI 中封装问答接口，调用 DeepSeek 大模型完成自然语言理解与回答生成，同时将实时路况、预警信息和路线建议拼接为上下文输入，确保回答贴合当前交通数据；该能力让系统从“看图表”升级为“可对话分析”。

(6). 流式输出与服务容错机制

后端通过 StreamingResponse 实现 SSE 流式推送，前端基于 ReadableStream 进行增量渲染，支持“正在思考”与逐段回答输出；当外部模型异常时，系统自动切换本地规则兜底回答，保证服务连续性与展示稳定性。

(7). 工程化部署与运行保障

项目采用 Docker + Docker Compose 进行容器化部署，统一编排 FastAPI 服务与 MySQL 数据库，降低环境差异带来的启动问题；同时保留本地 Python 环境部署路径，便于开发调试和后续上线扩展。

1.3 项目结构与使用指南

```
├─ backend/
│   ├── config.py
│   ├── database.py
│   ├── main.py
│   ├── models.py
│   ├── schemas.py
│   ├── routers/
│   │   ├── assistant.py
│   │   ├── insights.py
│   │   ├── roads.py
│   │   └── traffic.py
│   └── simulator/
│       ├── generator.py
│       ├── insights.py
│       └── scheduler.py
├─ frontend/
└─ assets/
```

```

| | └─ xuzhou_full.json
| └─ css/
| | └─ dashboard.css
| └─ js/
| | └─ api.js
| | └─ charts.js
| | └─ city-data.js
| | └─ detail.js
| | └─ insights.js
| | └─ main.js
| | └─ traffic-assistant.js
| | └─ xuzhou-map.js
| └─ detail.html
| └─ index.html
└─ scripts/
  └─ generate_history.py
  └─ init_db.sql
  └─ seed_roads.sql
└─ API_SERVICE_IMPLEMENTATION.md
└─ ARCHITECTURE.md
└─ docker-compose.yml
└─ Dockerfile
└─ main.py
└─ requirements.txt
└─ README.md

```

使用指南：

(1) 快速启动（Docker）

A. 启动容器

```
docker compose up -d --build
```

当前默认端口：

- 前端与后端服务：8000
- MySQL 宿主机端口：3307（容器内仍为 3306）

B. 导入路段种子数据（必须）

建议使用以下方式，避免 Windows 管道导致中文 SQL 编码异常：

```
docker cp ./scripts/seed_roads.sql traffic-mysql:/tmp/seed_roads.sql
```

```
docker exec traffic-mysql sh -c "mysql --default-character-set=utf8mb4 -uroot
-pwzh2004828 traffic_db < /tmp/seed_roads.sql"
```

C. 可选：生成历史数据（用于趋势图快速出数）

```
docker exec traffic-backend python scripts/generate_history.py
```

D. 访问地址

- 首页: <http://localhost:8000/>
- 图表详情页: <http://localhost:8000/detail.html>
- OpenAPI 文档: <http://localhost:8000/docs>
- 健康检查: <http://localhost:8000/api/health>

(2) 本地启动（不使用 Docker）

A. 初始化数据库

```
mysql -uroot -p < scripts/init_db.sql
```

```
mysql -uroot -p traffic_db < scripts/seed_roads.sql
```

B. 安装依赖

```
pip install -r requirements.txt
```

C. 设置环境变量

Windows PowerShell:

```
$env:DATABASE_URL="mysql+pymysql://root:your_password@127.0.0.1:3306/traffic_db?charset=utf8mb4"
```

D. 启动服务

```
uvicorn main:app --host 0.0.0.0 --port 8000
```

2. 可视化系统设计

2.1 概要设计

本项目的架构主要采用“四层架构”的设计思路，四层架构的核心思想是“界面与业务解耦、业务与依赖隔离”：表现层专注展示，应用层专注流程，领域层专注规则，基础设施层专注运行与集成。这样既能支撑课程演示场景的快速迭代，也便于后续替换真实数据源、优化算法模型和扩展部署规模。

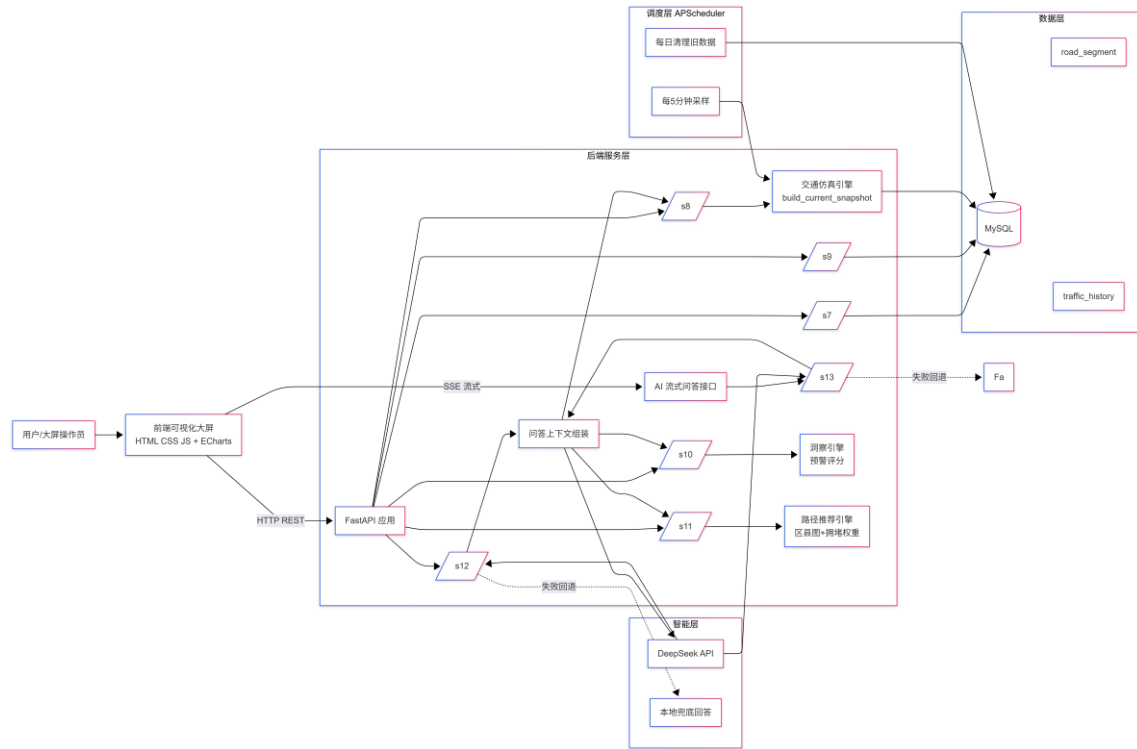


图 1. 系统架构图

(1). 表现层

表现层负责系统的人机交互与可视化呈现，核心任务是把后端返回的数据转换为可理解的业务界面。该层包含首页大屏、详情页、交通预警面板、绕行推荐面板和 AI 问答弹窗，支持实时刷新、趋势展示与流式文本渲染。实现上采用原生前端模块化组织，利用 ECharts 完成地图与图表绘制，使用 Fetch 获取普通接口数据，使用 ReadableStream/SSE 消费流式问答结果。该层不承担复杂业务计算，重点是状态管理、组件渲染和交互反馈。

(2). 应用层

应用层是系统的“用例编排中心”，负责把用户请求映射成可执行的业务流程。它统一管理 API 路由、参数校验、请求分发、响应封装以及异常处理，对外提供稳定接口规范。实现上以 FastAPI 路由模块为主体，按交通数据、洞察能力、AI 服务等场景进行拆分；通过 Pydantic 定义请求/响应模型，保证输入输出结构一致。该层本身尽量保持“薄”，主要做流程编排和协议适配，把具体业务规则下沉到领域层。

(3). 领域层

领域层承载系统的核心业务知识和决策逻辑，是项目价值的主要来源。该层

包括实时交通计算模型、交通预警评分模型、绕行推荐策略以及 AI 上下文构建与本地兜底策略。实现上采用纯 Python 业务模块，将算法逻辑与接口逻辑解耦：实时模块根据道路属性和时段特征生成当前状态，预警模块融合占有率/负载率/速度衰减/趋势进行风险判断，路径模块基于路网图和实时压力进行候选路径打分排序，AI 模块将多源交通数据组织成结构化提示词。该层强调可解释性、可维护性和可扩展性。

(4). 基础设施层

基础设施层负责为上层提供稳定的运行环境与外部能力接入，包括数据存储、任务调度、模型服务接入和部署运行。数据方面使用 MySQL 存储路段主数据与历史时序数据，使用 SQLAlchemy 进行 ORM 映射和会话管理；调度方面使用 APScheduler 周期性采样与历史清理；AI 能力通过 DeepSeek API 接入并由后端封装调用；部署方面通过 Docker 与 Docker Compose 实现服务容器化编排。该层的目标是把“外部依赖和运行细节”隔离在底部，保证上层业务逻辑可持续演进。

2.2 可视化设计

本项目主要由两大页面组成：主页面、图表详情页面。

(1). 主页面



图 2. 主页面

A. KPI 总览指标模块（总车流量、平均速度、整体拥堵指数）

该模块用于把多路段明细压缩成管理层可快速解读的三个核心指标。前端在 [charts.js](#) 中通过 `buildSummary` 进行计算：总车流量来自所有路段 `volume` 累加，平均速度按流量加权，拥堵指数由自由流速度均值与当前加权速度差值映射到 0 到 10 区间。交互链路是主页面每 5 秒请求一次 `/api/current` 和 `/api/roads`，由后端 [traffic.py](#) 返回实时快照，前端即时重绘数值。该模块体现了“后端提供路段级原子数据，前端做轻聚合展示”的设计。

B. 徐州地图态势联动模块

该模块的功能是将抽象交通数据空间化表达，回答“哪里更堵、哪里流量更高”。前端在 [xuzhou-map.js](#) 中先加载 [xuzhou_full.json](#) 注册地图，再把实时路段数据映射为“区县热度层 + 路段散点层”：区县热度按占有率均值计算，散点位置来自路段坐标映射，点大小与流量相关。后端通过 `/api/current` 提供每条路段的 `volume`、`speed`、`occupancy`，前端通过 ECharts `visualMap`、`scatter`、`tooltip` 实现联动渲染。技术组合是 GeoJSON + ECharts 地图能力 + 实时 API 拉取。

C. 交通预警模块

该模块用于输出“风险等级 + 原因 + 建议 + 预测值”，形成可执行提示。前端在 [insights.js](#) 中将预警结果渲染为卡片列表，展示 `risk_level`、`risk_score`、`reason`、`recommendation`、`predicted_speed`、`predicted_volume`；后端在 [insights.py](#) 的 `/api/alerts` 调用 [insights.py](#) 的评分函数，将当前快照与最近 1 小时历史融合计算。评分采用占有率、流量负载率、速度下降幅度和趋势因子加权，属于“实时态 + 趋势态”联合建模。技术上是 FastAPI + SQLAlchemy 历史查询 + Python 规则模型 + 前端卡片化渲染。

D. 绕行推荐模块

该模块的功能是从“看拥堵”升级为“给路径”，支持用户输入起终点后得到可执行路线建议。前端在 [insights.js](#) 绑定起终点选择器和刷新事件，触发 `/api/routes` 请求并渲染候选路线卡；后端在 [insights.py](#) 校验区县合法性后，调用 [insights.py](#) 的路网图算法：先 DFS 枚举路径，再根据区县拥堵压力动态调整耗时与风险，最后返回优先、平衡、备用三条方案。技术核心是“前端交互控件 + 后端图搜索与动态打分 + 结果可视化卡片”。

E. 主页面整体数据调度模块

该模块的功能是保障页面持续刷新与故障可恢复。前端在 [main.js](#) 采用分阶段加载、定时刷新和异常捕获机制，时钟 1 秒刷新、业务数据 5 秒刷新；后端接口拆分为 `current`、`alerts`、`routes` 三条服务线并行供给。这样形成了“单接口失败不拖垮全页”的容错交互，技术上依赖 `Promise` 并发、状态降级提示与模块化组件更新。

(2). 详情页面



图 3. 详情页面

A. 分析型版式编排模块

详情页的核心功能是将主页面的监控视角扩展为分析视角。页面结构在 [detail.html](#) 中采用“实时柱状图 + 24 小时趋势图 + 双仪表盘 + 滚动明细表”的分析型布局；前端入口 [detail.js](#) 统一驱动数据刷新和图表更新。后端继续复用 `/api/current` 与 `/api/trend` 两个接口，前后端通过“高频实时 + 低频趋势”双节奏协同，提升信息密度与可读性。

B. 实时路段流量柱状图模块

该模块用于横向比较不同路段当前流量。前端在 [charts.js](#) 通过 `setBarChart` 按路段生成柱体，颜色使用插值渐变表达相对强度；后端 `/api/current` 每次返回全路段快照，由前端每 5 秒重绘一次柱图。技术上是 `ECharts bar` + 自定义色彩插值算法 + `FastAPI` 实时接口。

C. 24 小时趋势曲线模块

该模块用于观察总流量时间演化，识别高峰、回落和异常波动。前端通过

setTrendChart 绘制平滑折线和面积渐变，后端 `/api/trend` 在 [traffic.py](#) 中从 `traffic_history` 按时间聚合 `volume`、`speed`、`occupancy` 并返回数组。前后端交互采用“前端 120 秒拉取一次 + 后端历史表聚合查询”的模式，技术栈是 SQLAlchemy 聚合 + ECharts line。

D. 速度与拥堵双仪表盘模块

该模块功能是用两个高可读指标刻画系统体征。前端基于 `buildSummary` 计算平均速度与拥堵指数后，使用双 `gauge` 组件并列展示；后端仍通过 `/api/current` 和 `/api/roads` 提供基础数据，前端进行口径统一后的仪表映射。技术上结合 ECharts gauge、阈值配色和动态数值动画，实现“趋势图看过程、仪表盘看结论”的互补。

E. 最近数据滚动表格模块

该模块用于保留明细可追溯能力。前端在 [charts.js](#) 中将最新路段记录渲染为表格并复制两份行数据形成无缝滚动，速度由行数动态调整；后端数据仍来自 `/api/current` 的标准字段。该设计把图形化展示和明细校验结合起来，技术上采用 DOM 表格渲染 + CSS 动画变量控制。

F. 详情页数据一致性保障模块

该模块保证趋势与实时数据口径一致。后端在 [scheduler.py](#) 每 5 分钟将当前仿真结果采样入库，供 `/api/trend` 使用；前端按不同刷新频率分别拉取 `current` 与 `trend`，避免高成本重算。技术上是 APScheduler 定时任务 + MySQL 时序存储 + 前端分频刷新策略。

(3). AI 助手



图 4. AI 助手

AI 助手是本项目在可视化大屏之外新增的语义交互层，其核心目标是把实时交通数据、预警结果和绕行方案转化为可理解、可执行的自然语言建议。前端采用弹窗式交互设计，在主页内提供快捷提问、连续对话和思考态展示，用户提问后由浏览器将问题与最近会话历史一并提交；后端接收请求后不会直接调用模型，而是先聚合当前路况、近一小时历史、风险预警和路径推荐等业务上下文，再通过大模型接口生成结构化回答（结论、依据、建议）。

在交互机制上，系统同时支持普通问答和流式问答：流式模式通过 SSE 持续推送思考步骤与答案增量，显著降低等待感；若模型调用失败或流式中断，则自动切换到本地规则兜底，保证服务连续可用。整体技术实现由前端原生 JavaScript 的消息状态管理与流式渲染、后端 FastAPI 的问答接口编排与上下文构造、以及模型侧的 DeepSeek 调用共同完成，形成了“数据可视化展示 + 数据语义解释”的闭环能力。

2.3 数据库设计

表一：road_segment

- 作用：存放每条路段的静态属性，不随时间频繁变化。

- 主要字段：
 - id: 主键，自增
 - name: 路段名称
 - type: 路段类型 (expressway / arterial / branch)
 - free_flow_speed: 自由流速度 (km/h)
 - capacity: 通行能力 (veh/h)

表二: traffic_history

- 作用: 存放每 5 分钟一条的历史采样数据, 用于趋势分析和预警计算。
- 主要字段：
 - id: 主键，自增
 - road_id: 外键，关联 road_segment.id
 - timestamp: 采样时间
 - volume: 流量 (veh/5min)
 - speed: 平均速度 (DECIMAL(5,2), km/h)
 - occupancy: 占有率 (DECIMAL(4,2), %)

3.可视化实现及验证

3.1 代码分析

(1). 主页面

A. KPI 总览指标模块 (总车流量、平均速度、整体拥堵指数)

前端:

```
export function buildSummary(roads, currentItems) {
  const totalVolume = currentItems.reduce((sum, item) => sum +
Number(item.volume || 0), 0);
  const weightedSpeed = totalVolume > 0
    ? currentItems.reduce((sum, item) => sum + Number(item.speed || 0) *
Number(item.volume || 0), 0) / totalVolume
    : 0;

  const roadSpeedBase = roads.length > 0
    ? roads.reduce((sum, road) => sum + Number(road.free_flow_speed || 0),
0) / roads.length
    : 0;
```

```

    const congestionIndex = clamp(10 * (roadSpeedBase > 0 ? 1 - weightedSpeed
/ roadSpeedBase : 0), 0, 10);

    return { totalVolume, averageSpeed: weightedSpeed, congestionIndex };
}
function updateKpis() {
    const summary = buildSummary(state.roads, state.currentItems);
    refs.totalVolume.textContent =
summary.totalVolume.toLocaleString("zh-CN");
    refs.averageSpeed.textContent = summary.averageSpeed.toFixed(2);
    refs.congestionIndex.textContent = summary.congestionIndex.toFixed(2);
}

```

后端：

```

@router.get("/current", response_model=CurrentTrafficResponse)
def get_current_traffic(db: Session = Depends(get_db)):
    roads = db.query(RoadSegment).order_by(RoadSegment.id.asc()).all()
    current_time = datetime.now()
    items = [build_current_snapshot(road, current_time) for road in roads]
    return {"timestamp": current_time, "items": items}
def calculate_current_traffic(road: Any, current_time: datetime) -> tuple[int,
float, float]:
    capacity = max(int(_get_value(road, "capacity")), 1)
    free_flow_speed = float(_get_value(road, "free_flow_speed"))
    base_flow = capacity * _base_flow_factor(current_time) * road_type_factor
* temporal_wave
    volume = int(round(max(0.0, np.random.normal(base_flow, noise_sigma))))
    congestion_ratio = min(0.9, volume / capacity)
    speed = max(0.0, free_flow_speed * (1 - congestion_ratio))
    occupancy = min(99.99, max(0.0, (volume / capacity) * 100 * uniform(0.8,
1.2)))
    return volume, round(speed, 2), round(occupancy, 2)

```

后端输出路段级实时快照，前端使用统一汇总函数构建 KPI。totalVolume 代表全局流量规模，averageSpeed 使用流量加权避免小流量路段干扰，congestionIndex 将速度损失映射到 0 到 10，方便与大屏配色和阈值联动。该模块体现“后端给原子数据，前端做展示口径聚合”的职责分离。

B. 徐州地图态势联动模块

前端：

```

async function loadXuzhouMap() {
    const response = await fetch("./assets/xuzhou_full.json");
    const geojson = await response.json();
}

```



```

    echarts.registerMap("xuzhou", geojson);
}

function buildDistrictData(roads, currentItems) {
    // 按区县聚合占有率
}

function buildRoadMarkers(currentItems) {
    return currentItems.map((item) => ({
        name: item.road_name,
        value: [...profile.coordinate, Number(item.volume) || 0],
        district: profile.district,
        volume: item.volume,
        speed: item.speed,
        occupancy: item.occupancy,
    }));
}

chart.setOption(buildMapOption(districtData, roadMarkers), true);

```

后端:

```

@router.get("", response_model=list[RoadSegmentRead])
def list_roads(db: Session = Depends(get_db)):
    return db.query(RoadSegment).order_by(RoadSegment.id.asc()).all()

@router.get("/current", response_model=CurrentTrafficResponse)
def get_current_traffic(db: Session = Depends(get_db)):
    roads = db.query(RoadSegment).order_by(RoadSegment.id.asc()).all()
    items = [build_current_snapshot(road, datetime.now()) for road in roads]
    return {"timestamp": datetime.now(), "items": items}

```

地图模块需要两类数据：路段静态信息（路段身份）和实时快照（动态数值）。前端先加载 GeoJSON 作为空间底图，再将实时数据映射成区县热度和散点叠层。这样既能展示区域级压力，也能查看单路段细节（流量、速度、拥堵），实现宏观与微观共存。

C. 交通预警模块

前端:

```

function renderAlerts(alerts) {
    if (!alerts.length) {
        alertList.innerHTML = renderEmptyState("当前没有高风险路段");
        return;
    }
    alertList.innerHTML = alerts.map(createAlertCard).join("");
}

```

```

}

function createAlertCard(alert) {
  return `
    <li class="alert-item">
      <span class="alert-level">${alert.risk_level}</span>
      <div class="alert-reason">风险分数 ${alert.risk_score} / 100,
      ${alert.reason}</div>
      <div class="alert-recommendation">${alert.recommendation}</div>
    </li>
  `;
}

```

后端：

```

@router.get("/alerts", response_model=TrafficAlertResponse)
def get_alerts(limit: int = Query(5, ge=1, le=10), db: Session = Depends(get_db)):
    roads = db.query(RoadSegment).order_by(RoadSegment.id.asc()).all()
    current_items = [build_current_snapshot(road, current_time) for road in roads]
    recent_history = db.query(TrafficHistory)...all()
    alerts = predict_traffic_alerts(roads, current_items, recent_history, current_time, limit=limit)
    return {"generated_at": current_time, "alerts": alerts}

risk_score = min(100.0, max(0.0,
    0.35 * float(current["occupancy"]) +
    0.35 * volume_ratio +
    0.2 * speed_drop_ratio +
    0.1 * trend_factor
))

```

后端先构造“当前态 + 近一小时趋势态”，再计算综合风险分并输出等级、原因、建议。前端不做二次推理，只做语义化渲染。这样保证风险判定口径统一，前端展示轻量且可控。

D. 绕行推荐模块

前端：

```

function bindRouteRefresh(handler) {
  routeRefresh.addEventListener("click", handler);
  routeOrigin.addEventListener("change", handler);
  routeDestination.addEventListener("change", handler);
}

```

```
function renderRoutes(routes) {
  if (!routes.length) {
    routeList.innerHTML = renderEmptyState("当前起终点暂无可路径");
    return;
  }
  routeList.innerHTML = routes.map(createRouteCard).join("");
}
const [alertsPayload, routesPayload] = await Promise.all([
  getAlerts({ limit: 5 }),
  getRoutes({ origin, destination }),
]);
```

后端:

```
@router.get("/routes", response_model=RouteRecommendationResponse)
def get_routes(origin: str = Query("鼓楼区"), destination: str = Query("铜山区"), db: Session = Depends(get_db)):
    if origin not in DISTRICT_OPTIONS:
        raise HTTPException(status_code=400, detail=f"Unsupported origin district: {origin}")
    if destination not in DISTRICT_OPTIONS:
        raise HTTPException(status_code=400, detail=f"Unsupported destination district: {destination}")

    current_items = [build_current_snapshot(road, current_time) for road in roads]
    options = recommend_routes(origin, destination, current_items)
    return {"generated_at": current_time, "origin": origin, "destination": destination, "options": options}
def recommend_routes(origin: str, destination: str, current_items: list[dict[str, Any]]) -> list[dict[str, Any]]:
    all_paths = _enumerate_paths(origin, destination)
    for path in all_paths:
        adjusted_minutes = float(edge["minutes"]) * (1 + pressure / 120.0)
        risk_score += pressure / 2.4 + float(edge["minutes"]) / 3.2
    ranked = sorted(scored_routes, key=lambda item: (item["estimated_minutes"], item["risk_score"]))
    return ranked[:3]
```

前端负责“输入起终点 + 触发刷新 + 渲染路径卡片”；后端负责“合法性校验 + 图搜索 + 动态打分”。推荐结果包含路径经过区县、走廊道路、预计耗时、风险分等字段，支持用户在同一视图中做路线比较。

E. 主页面整体数据调度模块

前端:

```

try {
    await refreshCurrent();
} catch (error) {
    setStatus("实时数据刷新失败，重试中", "error");
}

try {
    await refreshInsights();
} catch (error) {
    setStatus("预警或路线服务刷新失败，重试中", "error");
}

```

后端：

```

scheduler.add_job(
    sample_current_traffic,
    IntervalTrigger(minutes=5),
    id="traffic_sampling",
    replace_existing=True,
    kwargs={"session_factory": session_factory},
)

```

前端采用“分路刷新 + 分路容错”，某一模块失败不会影响其他模块；后端通过定时采样保持历史数据连续，支撑预警和趋势分析。两者共同形成展示层容灾与数据层可持续供给。

(2). 详情页面

A. 分析型版式编排模块

前端：

```

async function loadInitialData() {
    setStatus("正在加载数据", "loading");
    await refreshRoads();
    await Promise.all([refreshCurrent(), refreshTrend()]);
    setStatus("运行正常", "active");
}

function startTimers() {
    window.setInterval(async () => await refreshCurrent(), 5000);
    window.setInterval(async () => await refreshTrend(), 120000);
}

```

后端：

```

@router.get("/current", response_model=CurrentTrafficResponse)
@router.get("/trend", response_model=TrendResponse)

```

详情页采用双周期调度：实时数据高频刷新、趋势数据低频刷新，兼顾实时

性与稳定性。后端提供分离接口，前端按图表类型选择性刷新，避免不必要开销。

B. 实时路段流量柱状图模块

前端：

```
function setBarChart(roads, currentItems) {  
    const labels = roads.map((road) => road.name);  
    const values = roads.map((road) => itemByRoadId.get(road.id)?.volume ?? 0);  
    barChart.setOption(createBarOption(labels, values), true);  
}
```

后端：

```
items = [build_current_snapshot(road, current_time) for road in roads]  
return {"timestamp": current_time, "items": items}
```

前端柱状图只负责视觉映射（类目轴、颜色渐变、提示框）；后端负责生成路段级实时值。路段维度稳定（来自 roads），数值维度动态（来自 current），保证图形可读性和刷新连续性。

C. 24 小时趋势曲线模块

前端：

```
function setTrendChart(trend) {  
    const timestamps = trend.timestamps.map((value) => new Date(value));  
    trendChart.setOption(createTrendOption(timestamps, trend.volumes),  
true);  
}
```

后端：

```
query = db.query(  
    TrafficHistory.timestamp.label("timestamp"),  
    func.sum(TrafficHistory.volume).label("volume"),  
    func.avg(TrafficHistory.speed).label("speed"),  
    func.avg(TrafficHistory.occupancy).label("occupancy"),  
) .filter(...)  
  
rows =  
query.group_by(TrafficHistory.timestamp).order_by(TrafficHistory.timestamp.  
asc()).all()
```

后端负责时间窗口聚合并返回标准数组，前端直接映射为折线图。该设计避免前端做复杂时序处理，趋势口径统一且易复现。

D. 速度与拥堵双仪表盘模块

前端：

```
function setGaugeChart(summary) {
```

```
gaugeChart.setOption(createGaugeOption(summary), true);
}
```

双仪表盘并不直接依赖某个单独后端字段，而是由前端根据后端两类数据联合计算。这样可保持展示层的可调性（阈值、颜色、刻度）同时不污染后端业务接口。

E. 最近数据滚动表格模块

前端：

```
function renderRollingRows(items) {
  const rows = items.map((item) => `
    <tr>
      <td>${item.road_name}</td>
      <td>${item.volume}</td>
      <td>${Number(item.speed).toFixed(2)} km/h</td>
      <td>${formatDateTime(item.timestamp)}</td>
    </tr>`
  ).join("");
  return `${rows}${rows}`;
}

function setRollingTable(currentItems) {
  tableBody.innerHTML = renderRollingRows(currentItems);
  const duration = Math.max(12, Math.max(currentItems.length, 1) * 2.2);
  tableWrap.style.setProperty("--scroll-duration", `${duration}s`);
}
```

前端通过双份行拼接实现无缝滚动视觉，时间列直接展示后端时间戳，便于与图表同步核对，形成图形与数据明细的互证关系。

F. 详情页数据一致性保障模块

前端：

```
window.setInterval(async () => { await refreshCurrent(); }, 5000);
window.setInterval(async () => { await refreshTrend(); }, 120000);
```

后端：

```
scheduler.add_job(sample_current_traffic, IntervalTrigger(minutes=5), ...)
```

趋势数据不需要与实时数据同频刷新。前端按业务价值分配刷新频率，后端按 5 分钟沉淀历史点，两端节奏匹配，避免“高频请求低频数据”的资源浪费。

(3). AI 助手

前端：

```
async function sendQuestion(question) {
  // 1) 添加用户消息
```

```

    // 2) 创建 thinkingMessage + answerMessage
    // 3) 优先调用 streamAssistantMessage
    // 4) 流式失败时回退 sendAssistantMessage
    // 5) 双失败时本地 answerQuestion 兜底
}

export async function streamAssistantMessage({ question, history = [], onEvent }
= {}) {
    const response = await fetch(buildUrl("/assistant/chat/stream"), {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ question, history }),
    });

    const reader = response.body.getReader();
    // 逐块解析 SSE: thinking / thinking_delta / answer_delta / done
}

```

后端:

```

@router.post("/chat", response_model=TrafficChatResponse)
def chat(request: TrafficChatRequest, db: Session = Depends(get_db)):
    context = _build_context(db, question, history)
    try:
        answer = _call_deepseek(question, str(context["context"]), history)
    except Exception:
        answer = _fallback_answer(question, context)

@router.post("/chat/stream")
def chat_stream(request: TrafficChatRequest, db: Session = Depends(get_db)):
    for step in _build_thinking_steps(question):
        yield _sse_event({"type": "thinking", "text": step})

    for event in _iter_deepseek_stream(...):
        # thinking_delta / answer_delta

    yield _sse_event({"type": "done", "provider": provider, "model":
model, ...})

def _build_context(db: Session, question: str, history: list[dict[str, str]]
| None = None) -> dict[str, object]:
    roads = db.query(RoadSegment).order_by(RoadSegment.id.asc()).all()
    current_items = [build_current_snapshot(road, current_time) for road in
roads]

    recent_history = db.query(TrafficHistory).filter(...).all()
    alerts = predict_traffic_alerts(...)

```

```
route_options = recommend_routes(origin, destination, current_items)
# 组装文本上下文供模型调用
```

该模块采用“前端会话状态机 + 后端数据增强问答”协同模式：

- 前端负责交互体验（弹窗、消息、思考态、流式渲染、快捷提问、自动聚焦）。
- 后端负责语义可信度（问题实体识别、交通上下文构造、模型提示词约束、异常兜底）。
- 流式协议将模型输出拆分为思考增量与答案增量，使用户在等待中持续获得反馈。
- 双重降级链路（流式失败 -> 普通问答 -> 本地规则）确保任何网络或模型波动下都能返回可读答案。

最终，AI 助手把主页面与详情页面的数据能力汇聚成一个“问答入口”，显著降低用户获取结论的认知成本。

3.2 运行截图

(1). 主页面



图 5. 主页面

(2). 详情页面



图 6. 详情页面

(3). AI 助手

输入“告诉我今天的交通情况”：

图 7. 输入问题

查看回答情况，可以看到系统先输出思考过程：

告诉我今天的交通情况

你 - 16:50

思考完成

- 正在建立交通问题分析上下文...
- 正在解析问题意图（路况查询/路线推荐/风险预警）...
- 正在提取关键实体（区县、路段、时段）...
- 正在读取实时路况、预警和绕行候选...
- 未识别到明确区县，正在按全局交通态势进行推理...
- 正在组织结构化回答（结论、依据、建议）...

AI 助手 - 16:50

图 8. 查看思考内容

查看回答内容：

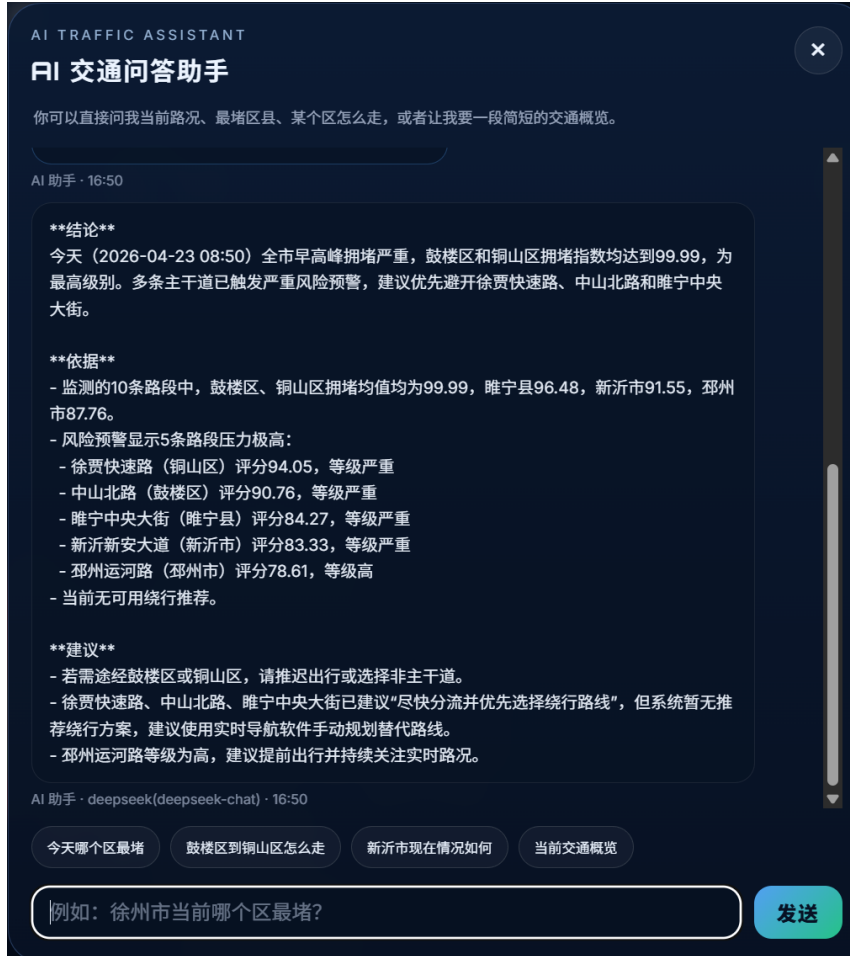


图 9. 查看 AI 回答的内容

4.总结及体会

本项目采用前后端协同的分层实现：后端使用 FastAPI 提供四类核心接口（道路、实时与趋势、预警与路线、AI 助手），数据层使用 SQLAlchemy 映射路段主表与历史表，调度层使用 APScheduler 做 5 分钟采样入库和每日历史清理；前端采用原生 ES Module 架构，主页与详情页分离编排，ECharts 分别承载地图、柱状图、趋势图、双仪表盘和滚动表。业务算法上，实时流量由路段容量、自由流速度、道路类型、时段因子和随机扰动共同生成；预警分数采用多因子加权模型；绕行推荐基于区县路网图的路径枚举与动态加权排序（时间与风险双目标）。AI 模块不是独立聊天，而是先聚合实时路况、预警和路线上下文，再调用模型生成结构化回答，并提供流式 SSE 输出与本地兜底，最终形成“监

控-分析-解释-建议”的闭环系统。

最大的技术收获是“接口协议先行”与“口径统一优先”：前后端通过明确的请求响应模型约束字段，显著降低了联调成本和展示歧义。第二个收获是性能与实时性平衡，实时数据和趋势数据采用分频刷新策略，避免高频重绘低价值数据；前端通过局部更新图表而非整页重渲染，后端通过任务调度保障历史连续性。第三个收获是工程稳定性建设，包括接口重试、超时控制、状态提示、流式失败回退、模型失败降级、本地规则兜底等。实践中还验证了很多“非业务但关键”的细节：例如 Docker 端口冲突处理、SQL 导入编码问题、依赖安装网络超时处理，这些都直接决定系统能否稳定运行。

本项目我采用的是 vibecoding 的方式完成的，vibecoding 在这个项目里最有价值的地方是快速迭代技术方案，而不是一次性生成终稿。具体体现在：先搭主干（接口与页面骨架），再逐步补业务算法，再增强交互体验（SSE 思考态），最后做部署和文档固化。这个过程要求持续的技术校验：每次改动都要回到接口行为、数据口径、异常链路和页面刷新逻辑做验证，否则很容易出现“看起来能跑但结果不可靠”。我的体会是，把 AI 当作高效的实现与重构工具最有效，但关键技术决策仍需人工把关，尤其是模型上下文设计、评分公式权重、刷新策略、降级机制和部署参数。只有“AI 提速 + 人工验收”结合，才能在开发效率和系统质量之间取得真正平衡。